Comprendre l’ATmega328P

Des bits au programme qui commande une LED

Ce cahier AVR* rassemble tes notes dans un ordre progressif. Les explications sont en français, les commentaires de code en anglais et tes huit images sont conservées dans leurs sections.

Le fil conducteur
Le programme manipule des bits dans des registres pour lire un bouton et commander une LED. Les timers mesurent le temps ; les mémoires conservent le programme et ses données.

Je veux comprendre…	Je vais à…
Les bits, les nombres et les masques	Pages 2 à 4
Les broches, les ports et les registres	Pages 5 et 6
Le bouton et la résistance pull-up	Pages 7 à 11
Un nouvel appui et les rebonds	Pages 12 à 14
L’horloge, les timers et ICP1	Pages 15 à 17
Les interruptions	Page 18
La flash, la SRAM et l’EEPROM	Pages 19 à 21
Retrouver les sections de la datasheet	Page 22

Le matériel utilisé dans les exemples

Les exemples principaux utilisent **PB0 pour la LED** et **PD2 pour le bouton**, avec une pull-up et un bouton relié à GND. L’alimentation est supposée à 5 V et l’horloge à 16 MHz lorsque ces valeurs sont nécessaires.

Ces noms identifient des broches du microcontrôleur* ; les noms D1 et SW1 viennent du schéma de ta carte. Le câblage réel doit correspondre à l’exemple.

Lire le cahier
Un astérisque signale un mot technique expliqué en une phrase en bas de page. Les références reprennent les titres de ta datasheet, version 7810D–AVR–01/15.

* AVR : architecture et famille de microcontrôleurs dont fait partie l’ATmega328P.
* Microcontrôleur : puce qui réunit un processeur, de la mémoire et des périphériques pour piloter un système électronique.

Un nombre composé de bits

Un **bit** prend la valeur 0 ou 1. Ce sont deux états logiques ; dans un circuit, ils sont représentés par des niveaux électriques. Huit bits forment un **octet**.

Le bit le plus à droite est le **bit 0**. Chaque position possède un poids égal à une puissance de deux : pour lire le nombre, on additionne les poids des bits à 1.

Bit	7	6	5	4	3	2	1	0
Poids	128	64	32	16	8	4	2	1
Valeur	0	0	0	0	1	0	1	0

Exemple : 00001010
Les bits 3 et 1 valent 1 : 8 + 2 = 10. Le nombre binaire 00001010 représente donc le nombre décimal 10.

Binaire sur 8 bits	Calcul	Décimal
00001101	8 + 4 + 1	13
00011001	16 + 8 + 1	25
00101101	32 + 8 + 4 + 1	45
11111111	128 + 64 + 32 + 16 + 8 + 4 + 2 + 1	255

Passer du décimal au binaire

Pour écrire 13, décompose-le en 8 + 4 + 1, puis mets à 1 les positions correspondantes : **00001101**. Les autres positions valent 0.

Un `uint8_t`* contient un entier non signé* sur 8 bits : de 0 à 255. Il nécessite `<stdint.h>`. Si ce fichier n'est pas autorisé dans ton exercice, un `unsigned char` sur cet AVR permet aussi de stocker un octet.

```
uint8_t nb = 13;  // Store an unsigned 8-bit value: 13.
```

* uint8_t : type C défini par stdint.h pour représenter un entier non signé sur exactement huit bits.
* Non signé : type entier qui ne représente pas de valeurs négatives.

Les opérateurs qui travaillent sur les bits

Opérateur	Rôle	Exemple
& — AND	Un bit vaut 1 seulement si les deux bits comparés valent 1.	1 & 0 = 0
| — OR	Un bit vaut 1 si au moins un des deux bits vaut 1.	1 | 0 = 1
^ — XOR*	Un bit vaut 1 si les deux bits sont différents.	1 ^ 1 = 0
~ — complément	Inverse chaque bit de l'opérande entier.	Sur 8 bits : 00000100 devient 11111011
<< — décalage à gauche	Déplace les bits vers des positions de poids plus élevé.	1 << 2 = 4
>> — décalage à droite	Déplace les bits vers des positions de poids plus faible.	8 >> 1 = 4

Comparer AND, OR et XOR

A	B	A & B	A | B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Pour les valeurs positives utilisées ici, décaler d'un bit à gauche ressemble à multiplier par 2, tant que la valeur reste représentable. Décaler un entier non signé à droite divise par 2 en supprimant la partie fractionnaire.

Ne pas confondre & et &&
& compare les bits d'entiers. && combine deux conditions logiques. De même, | travaille bit par bit, tandis que || signifie « ou » entre deux conditions.

Le schéma `~00000100 = 11111011` montre seulement huit positions. En C, les promotions entières* peuvent faire le calcul sur un type plus large : sur cet AVR, `1 << 2` est normalement calculé comme un `int` de 16 bits.

* XOR : opération qui produit 1 lorsque les deux bits comparés sont différents, ce qui permet d'inverser un bit ciblé.
* Promotion entière : conversion automatique d'un petit type entier vers int ou unsigned int avant certaines opérations C.

Un masque pour choisir un seul bit

Un **masque*** sélectionne une ou plusieurs positions. `1 << n` place un 1 au bit n : `1 << 0` vaut 1, `1 << 2` vaut 4 et `1 << 5` vaut 32.

Je veux…	Écriture C	Effet
Mettre le bit n à 1	registre |= (1 << n);	Les autres bits restent inchangés.
Mettre le bit n à 0	registre &= ~(1 << n);	Les autres bits restent inchangés.
Inverser le bit n	registre ^= (1 << n);	0 devient 1 et 1 devient 0.
Tester le bit n	registre & (1 << n)	Le résultat est nul si ce bit vaut 0.

Exemple complet sur huit positions

Étape	Bits affichés	Valeur
Départ	10110001	177
Masque du bit 2	00000100	4
Après OR avec ce masque	10110101	181
Masque du bit 5 inversé sur 8 bits	11011111	223
Après AND : bit 5 effacé	10010101	149

Tester PD2 donne 0 ou 4, pas 0 ou 1
PD2 est le bit 2. Ainsi, PIND & (1 << PD2) vaut 0 si PD2 est LOW, ou 4 si PD2 est HIGH. Dans une condition C, toute valeur non nulle est vraie.

```
// Test only bit 2: a zero result means PD2 is LOW.
if ((PIND & (1 << PD2)) == 0)
{
    // Handle the LOW state here.
}
```

Les parenthèses sont importantes : écris bien `(PIND & masque) == 0`. « Non nul » signifie différent de zéro ; c’est le test naturel pour ce résultat masqué.

* Masque : valeur binaire utilisée pour sélectionner les bits que l’on veut tester ou modifier.

La broche physique et le bit du registre

Une **broche**, ou pin, est une connexion électrique du microcontrôleur. Tu y relies une LED, un bouton ou un capteur par les fils ou les pistes de la carte. **PB0** signifie broche 0 du port B ; **PD2** signifie broche 2 du port D.

Un **port** est un groupe de broches GPIO*. Un **registre*** est une case matérielle que le code peut lire ou écrire pour configurer ou commander le microcontrôleur.

Registre	Fonction	Valeur du bit n
DDRx	Choisir la direction de la broche du port x.	0 : entrée ; 1 : sortie
PORTx	Piloter une sortie ou sélectionner la pull-up d'une entrée.	Sortie : 0 LOW / 1 HIGH ; entrée : 1 active la pull-up si elle est autorisée
PINx	Lire le niveau logique réellement détecté sur les broches.	0 : LOW ; 1 : HIGH

Pour le port B, ces noms deviennent `DDRB`, `PORTB` et `PINB`. Pour le port D : `DDRD`, `PORTD` et `PIND`. Le bit 0 correspond à la broche 0 de ce port, le bit 1 à la broche 1, etc.

Mode	Sens de l'information	Exemple
INPUT — entrée	Composant extérieur → microcontrôleur	Lire un bouton
OUTPUT — sortie	Microcontrôleur → composant extérieur	Commander une LED

Deux précisions utiles
PINx peut lire une broche même lorsqu'elle est configurée en sortie. Sur l'ATmega328P, écrire un 1 dans un bit de PINx inverse le bit correspondant de PORTx : lire et écrire ce registre ne font donc pas la même chose.

PB6 et PB7 servent souvent à l'oscillateur externe sur les cartes à 16 MHz : ne les prends pas comme exemples de LED sans vérifier le schéma. VCC et GND sont des broches d'alimentation, pas des sorties à configurer avec DDRx.

Datasheet : 13.2.1 Configuring the Pin ; 13.2.2 Toggling the Pin ; 13.2.4 Reading the Pin Value.

* Registre : emplacement matériel accessible au programme pour lire un état, stocker une valeur de travail ou configurer un périphérique.
* GPIO : General Purpose Input/Output, broche utilisable comme entrée ou sortie numérique selon sa configuration.

Configurer PB0 et PD2, puis lire un niveau

`#include <avr/io.h>` fournit les noms des registres et des bits du bon AVR. Le modèle est choisi à la compilation avec `-mmcu=atmega328p` ; l'inclusion ne configure pas elle-même les broches.

```
#include <avr/io.h>

// DDRB controls pin direction: set bit 0 to 1 for output.
DDRB |= (1 << PB0);

// DDRD controls pin direction: clear bit 2 for input.
DDRD &= ~(1 << PD2);

// PORTD on an input: set bit 2 to enable its pull-up.
PORTD |= (1 << PD2);

// PORTB on an output: set bit 0 to drive PB0 HIGH.
PORTB |= (1 << PB0);

// PORTB on an output: clear bit 0 to drive PB0 LOW.
PORTB &= ~(1 << PB0);
```

Ces lignes illustrent les opérations et doivent être placées dans une fonction pour être exécutées. Chaque commentaire explique **le rôle du registre** et **la valeur assignée au bit ciblé** ; les autres bits sont conservés.

Numérique et analogique

Lecture numérique	Mesure analogique
Reconnaître LOW ou HIGH, donc un bit 0 ou 1.	Mesurer une tension puis la convertir en une valeur numérique.
PIND permet de lire les niveaux du port D.	L'ADC* de l'ATmega328P produit un résultat sur 10 bits : 0 à 1023.
Pratique pour un bouton.	Pratique pour un potentiomètre ou certains capteurs.

En sortie, HIGH est une tension proche de VCC* et LOW une tension proche de GND*. En entrée, le matériel compare la tension à des seuils : une tension intermédiaire n'a pas un état numérique garanti. La plage mesurée par l'ADC dépend de sa tension de référence.

Tension et courant
Une sortie HIGH impose un niveau de tension ; le courant dépend du composant connecté. Une LED doit être accompagnée d'une résistance adaptée, et HIGH ne signifie LED allumée que si son câblage le prévoit.

* ADC : convertisseur analogique-numérique qui transforme une tension d'entrée en un nombre selon une tension de référence.
* VCC / GND : VCC désigne l'alimentation positive et GND la référence électrique de 0 V.

Bouton relâché : PD2 reste HIGH

Dans ce montage, une résistance pull-up* relie PD2 à VCC. Le bouton est relié à GND et, au repos, ses contacts sont séparés : le chemin vers GND est ouvert.

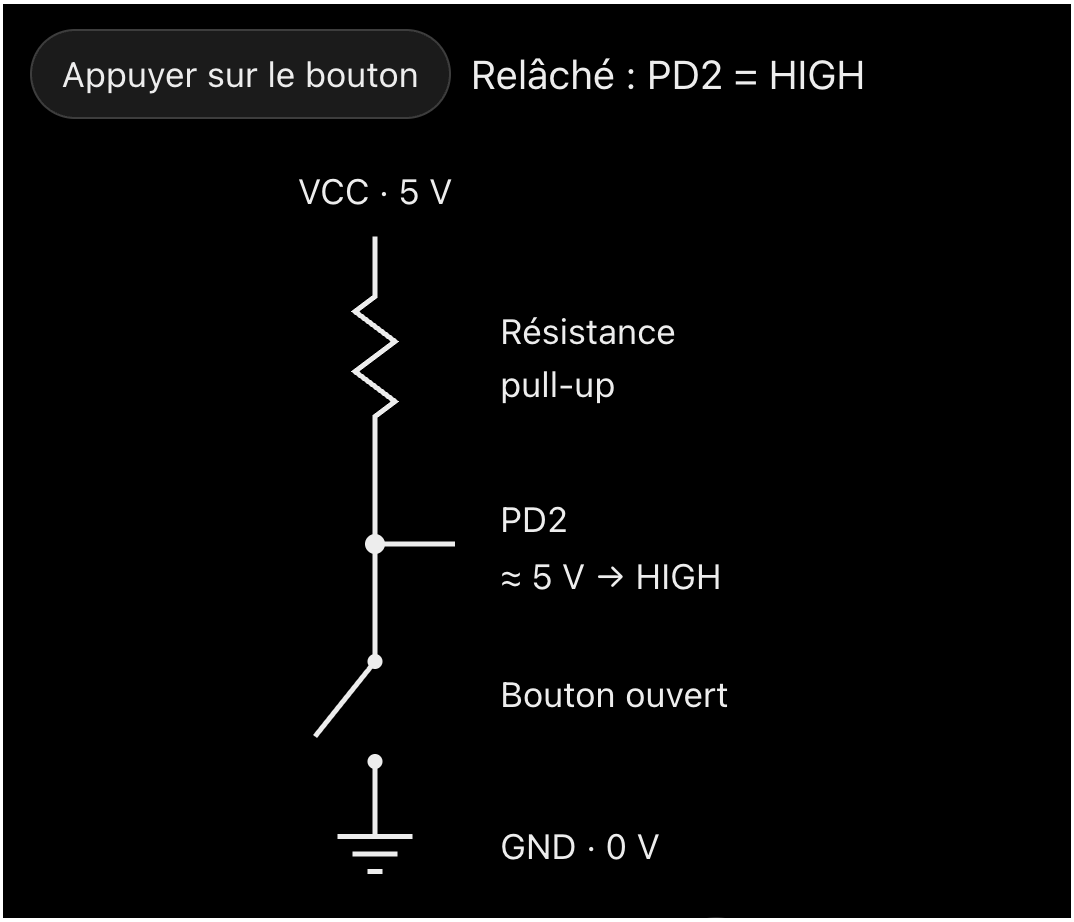


Image originale 1 — Bouton relâché, pull-up présente : PD2 est proche de 5 V.

Ce qui se passe	Ce que le microcontrôleur lit
Le bouton ne relie pas PD2 à GND.	La pull-up maintient PD2 près de VCC.
PD2 est à un niveau haut défini.	Le bit 2 de PIND vaut 1 : HIGH.

Un chemin ouvert n’est pas une tension nulle
Le courant ne passe pas par le bouton ouvert, mais la tension* de PD2 peut quand même être de 5 V grâce à la pull-up. Tension et courant sont deux grandeurs différentes.

* Pull-up : résistance qui relie une entrée à l’alimentation positive pour lui donner un niveau HIGH lorsque rien d’autre ne la force à LOW.
* Tension : différence de potentiel électrique entre deux points, exprimée en volts et mesurée ici par rapport à GND.

Bouton appuyé : PD2 passe LOW

Quand tu appuies, les contacts se touchent. Le bouton ferme le chemin entre PD2 et GND, où un courant* peut alors circuler à travers la résistance : la tension sur PD2 descend près de 0 V, malgré la pull-up.

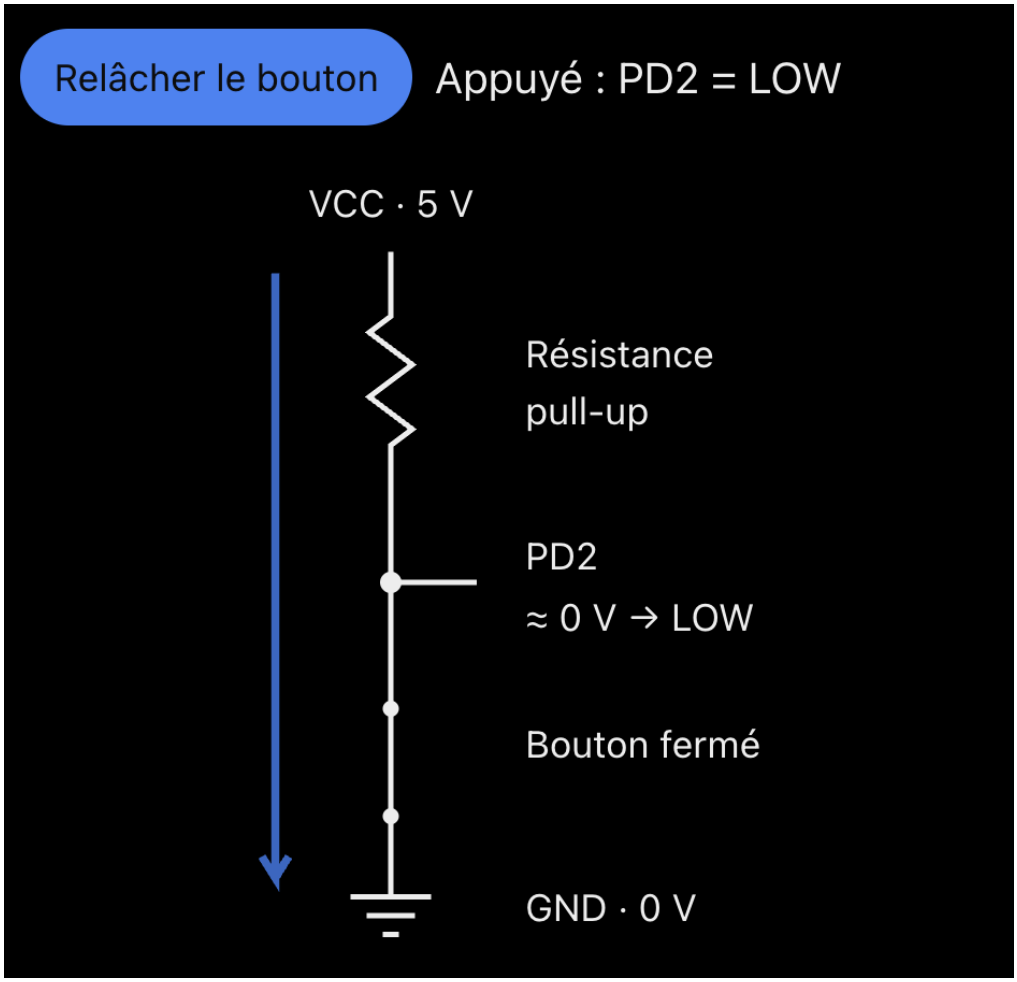


Image originale 2 — Bouton fermé : le chemin de courant passe par la résistance puis le bouton.

État physique	Interprétation
PD2 reliée à GND	Le bit 2 de PIND vaut 0 : LOW.
Courant de VCC vers GND à travers la résistance	La résistance limite ce courant ; PIND lit la tension sur PD2.

Le bouton est actif à LOW*
Pour ce câblage : appuyé signifie LOW et relâché signifie HIGH. La fonction isPressed() peut transformer une lecture masquée égale à 0 en une valeur logique vraie.

Le bouton n’alimente pas directement la LED : il fournit une information au microcontrôleur, et le programme commande ensuite la sortie de la LED.

* Courant : déplacement de charges électriques dans un circuit fermé, exprimé en ampères.
* Actif à LOW : convention dans laquelle l’état actif d’un signal est représenté par un niveau logique bas.

Sans pull-up, une entrée peut flotter

Si le bouton est relâché et qu’il n’y a ni pull-up ni autre connexion qui fixe le niveau, PD2 n’est reliée clairement à VCC ou à GND. On dit que l’entrée est **flottante***.

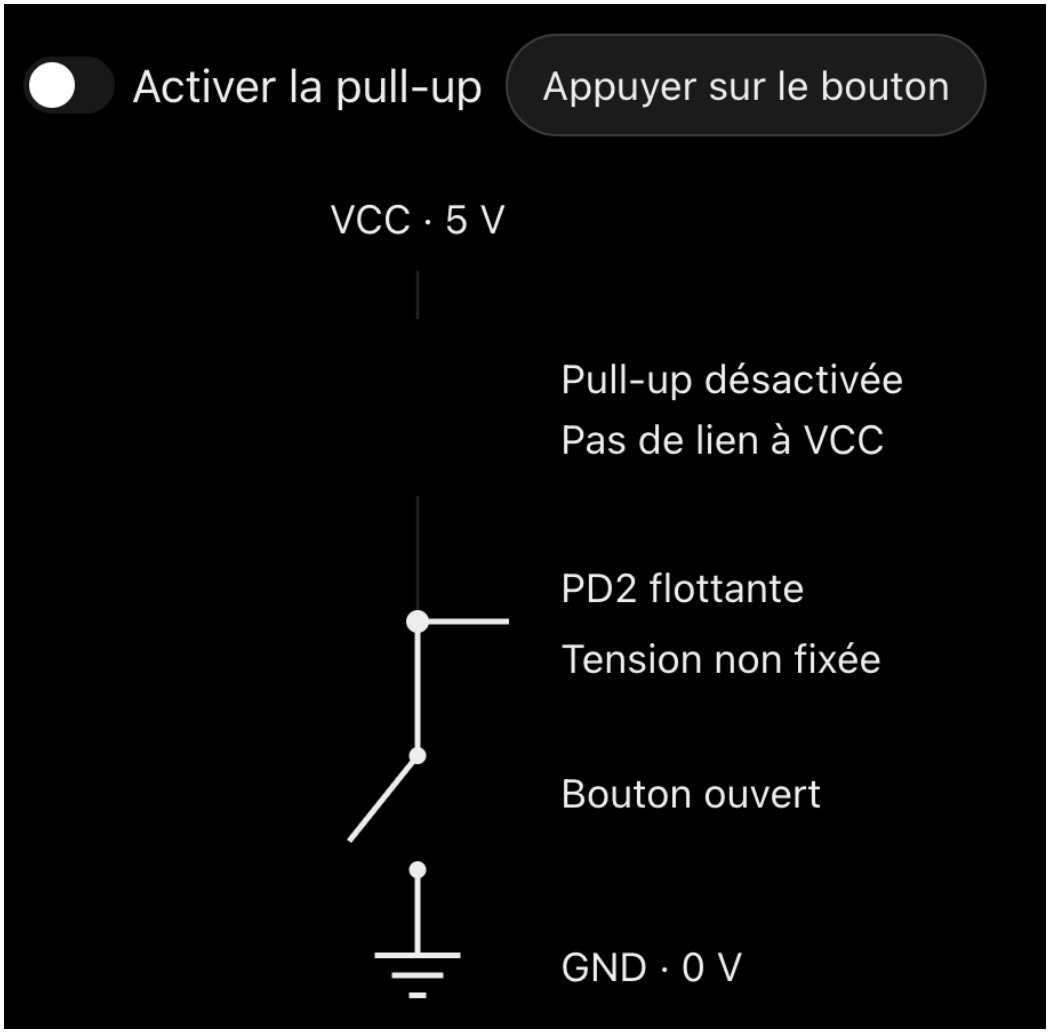


Image originale 3 — Pull-up désactivée et bouton ouvert : la tension de PD2 n’est pas définie.

Pourquoi la lecture devient incertaine
Des perturbations électriques*, des couplages et des parasites ou des charges résiduelles peuvent modifier la tension de l’entrée. Elle peut être lue HIGH ou LOW sans que le bouton ait changé de position.

Le mot « fixée » signifie **maintenue près d’une valeur connue**, et non bloquée pour toujours. Le niveau peut ensuite changer volontairement lorsque tu appuies.

Pour une broche, emploie « flottante ». Le mot « volatile » décrit plutôt une mémoire dont les données disparaissent sans alimentation.

* Entrée flottante : entrée dont la tension n’est pas imposée par une connexion suffisante à un niveau électrique défini.
* Perturbation électrique : variation indésirable d’un signal, susceptible de modifier le niveau logique lu par le circuit.

Sans pull-up, appuyer donne quand même LOW

Même sans pull-up, le bouton appuyé relie directement PD2 à GND. La tension est alors définie, ce qui donne un état stable* : le problème de ce montage apparaît surtout quand le bouton est relâché.

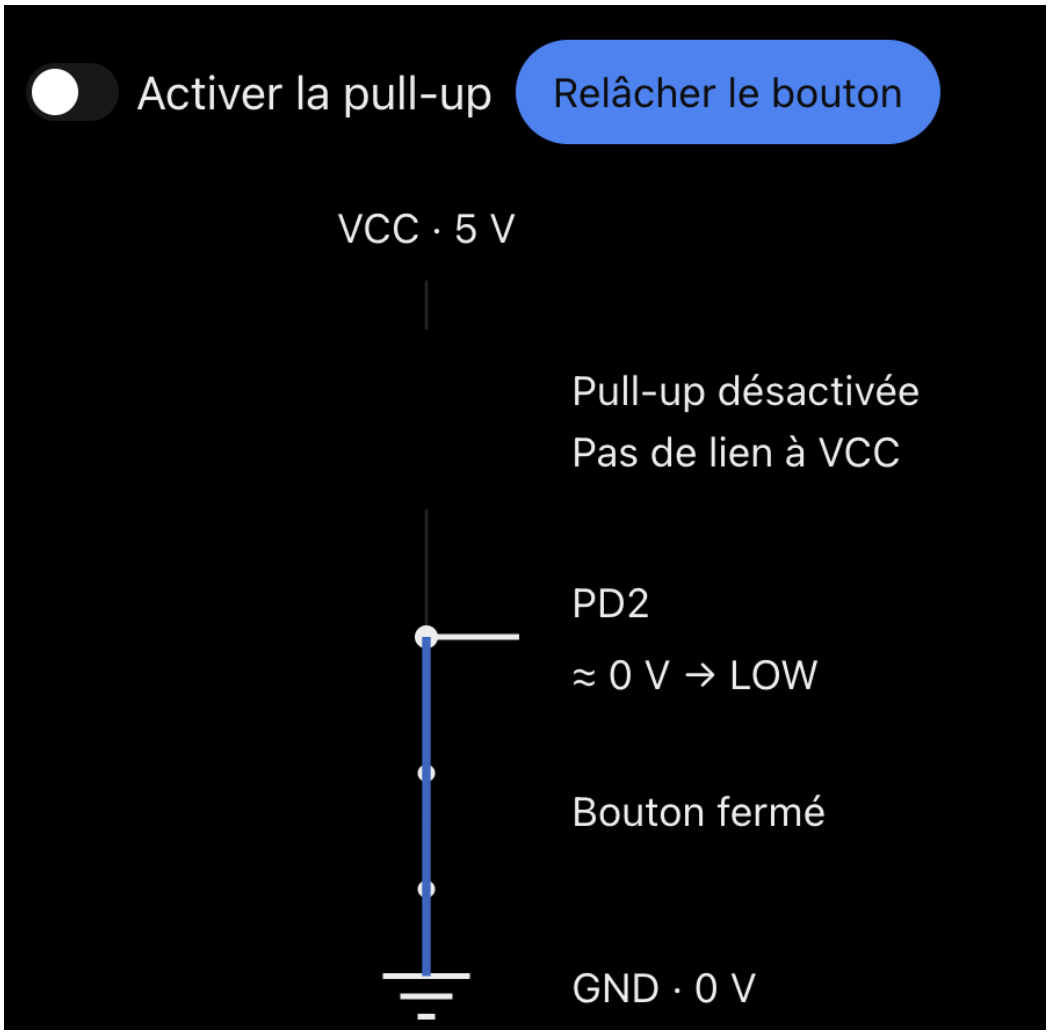


Image originale 4 — Pull-up désactivée et bouton fermé : PD2 est reliée à GND.

Sans pull-up	Résultat
Bouton relâché	Entrée flottante : niveau non garanti.
Bouton appuyé	Entrée reliée à GND : LOW.

La pull-up donne un repos reconnaissable
Avec la pull-up, HIGH représente clairement le repos. Sans elle, une lecture LOW à bouton relâché pourrait provenir d’une tension indéfinie plutôt que d’un véritable appui.

* État stable : niveau électrique suffisamment défini pour être lu de façon fiable, sans empêcher un changement volontaire du signal.

Avec la pull-up : deux états bien définis

La pull-up* établit HIGH au repos ; le bouton fermé impose LOW pendant l’appui. Ces deux images montrent le même circuit dans ses deux positions.

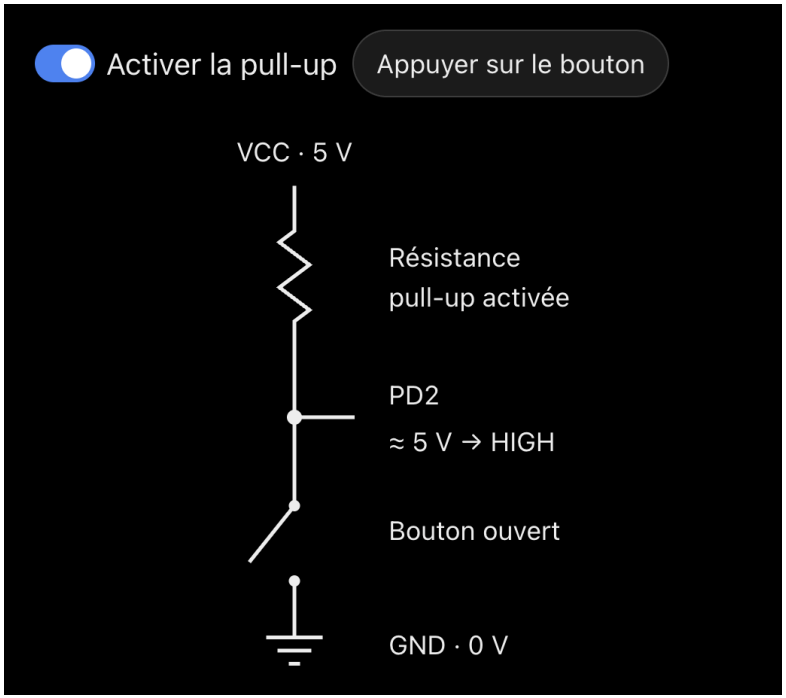


Image originale 5 — Relâché : HIGH.

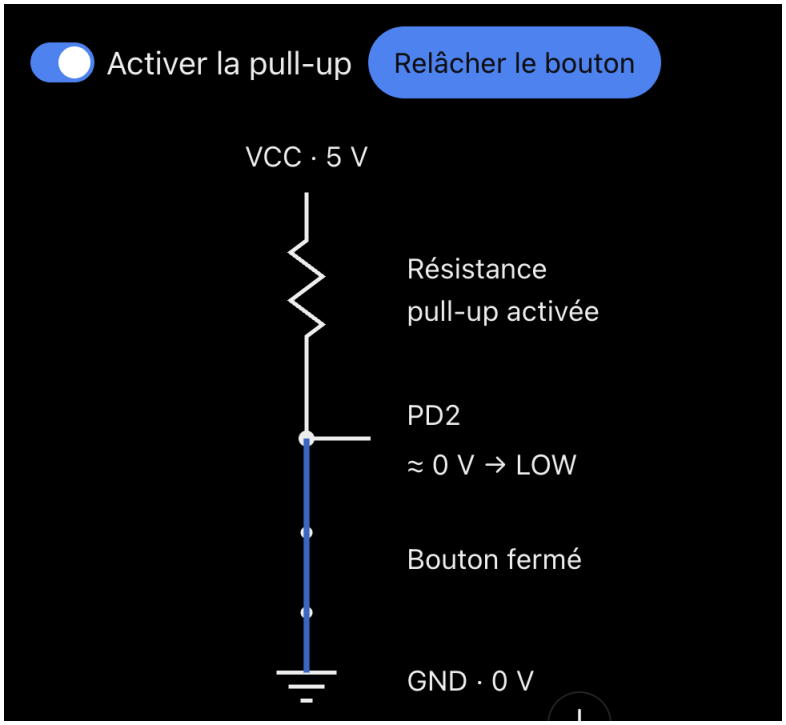


Image originale 6 — Appuyé : LOW.

Configuration de l’entrée
DDRD, bit 2 = 0 : entrée. PORTD, bit 2 = 1 : pull-up interne* sélectionnée, à condition que le bit global PUD* ne désactive pas les pull-up.

* Pull-up interne : résistance intégrée au microcontrôleur que le programme peut sélectionner pour une broche configurée en entrée.
* PUD : bit global Pull-up Disable qui, lorsqu’il vaut 1, désactive les résistances pull-up internes.

Reconnaître un nouvel appui

Lire « appuyé » ne suffit pas pour inverser une LED une seule fois par appui : la boucle peut relire cet état des milliers de fois. Il faut reconnaître le passage **relâché → appuyé**, appelé un front*.

État précédent	État actuel	Interprétation
Relâché — false	Relâché — false	Toujours au repos.
Relâché — false	Appuyé — true	Nouvel appui : événement à traiter.
Appuyé — true	Appuyé — true	Appui maintenu : pas un nouvel appui.
Appuyé — true	Relâché — false	Relâchement : le prochain appui pourra être reconnu.

```
// Detect a newly pressed button, not a held button.
if (buttonPressed && !buttonPreviouslyPressed)
{
    // Handle this new press here.
}

// Keep the accepted state for the next loop.
buttonPreviouslyPressed = buttonPressed;
```

Le bouton et la LED ont deux états différents
buttonPressed indique l'état accepté du bouton ; buttonPreviouslyPressed mémorise le tour précédent ; ledIsOn représente l'état souhaité de la LED. Utilise pressed/released pour le bouton et on/off pour la LED.

Avec le bouton relié à GND	Nom du changement électrique
Tu appuies : HIGH → LOW	Front descendant.
Tu relâches : LOW → HIGH	Front montant.

Dans une fonction qui reçoit un pointeur*, `&variable` transmet son adresse et `*parametre` accède à sa valeur. Modifier le pointeur seul ne modifie pas automatiquement la variable dans main.

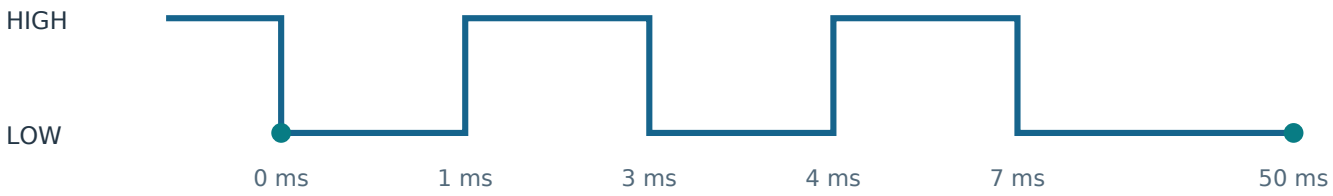
bool, true et false nécessitent normalement <stdbool.h> en C avec ton GCC 9. Si cet en-tête n'est pas autorisé, tu peux représenter ces états par un unsigned char : 0 pour faux et 1 pour vrai.

* Front : transition d'un signal numérique entre LOW et HIGH, montante de LOW à HIGH ou descendante de HIGH à LOW.
* Pointeur : variable qui contient une adresse mémoire et permet d'accéder à l'objet situé à cette adresse.

Un appui peut produire plusieurs changements

Les contacts mécaniques peuvent se toucher, se séparer brièvement, puis se toucher à nouveau avant de rester en place. Cet **effet de rebond*** apparaît à l'appui et peut aussi apparaître au relâchement.

Un seul appui physique · plusieurs fronts descendants



Exemple illustratif · temps non à l'échelle

Temps illustratif	Contacts	Lecture de PD2
0 ms	Se touchent une première fois.	LOW
1 ms	Se séparent brièvement.	HIGH
3 ms	Se touchent à nouveau.	LOW
4 ms	Se séparent encore.	HIGH
7 ms	Restent en contact.	LOW
50 ms	Toujours en contact.	LOW

Une boucle rapide peut reconnaître plusieurs fronts descendants pendant un seul appui. Si chacun inverse la LED, un appui physique peut provoquer plusieurs inversions.

Pull-up et anti-rebond ont des rôles différents
La pull-up donne un niveau défini quand le bouton est ouvert. L'anti-rebond empêche les changements temporaires des contacts d'être pris pour plusieurs actions : la pull-up ne supprime pas ces rebonds.

Les temps du tableau sont fictifs : la durée réelle des rebonds dépend du bouton. La datasheet décrit la lecture et les fronts, pas une durée universelle de rebond.

* Rebond mécanique : alternance rapide entre contact établi et contact ouvert lors d'un appui ou d'un relâchement.

Deux lectures séparées par un délai

L’idée de ton anti-rebond* est de lire le bouton, attendre un court délai puis relire. Si les deux lectures, ou échantillons*, sont identiques, tu acceptes cet état ; sinon, tu conserves le dernier état accepté.

```
// Read PD2: a masked value of zero means pressed.
beforeDelay = isPressed(PIND & (1 << PD2));

// Wait briefly before checking the button again.
_delay_ms(50);

// Read PD2 again after the waiting period.
afterDelay = isPressed(PIND & (1 << PD2));

// Accept matching samples; otherwise keep the old state.
if (beforeDelay == afterDelay)
    buttonPressed = beforeDelay;
```

Première lecture	Seconde lecture	Décision
Appuyé	Appuyé	Accepter appuyé.
Relâché	Relâché	Accepter relâché.
Appuyé	Relâché	Garder l’ancien état accepté.
Relâché	Appuyé	Garder l’ancien état accepté.

Ce que le délai fait réellement
Les contacts peuvent continuer à rebondir pendant _delay_ms(50), mais le code de cette boucle n’analyse pas les changements intermédiaires. Le délai ne répare pas physiquement le bouton.

Initialise l’état accepté avant la boucle, puis mémorise cet état à la fin du tour. Applique la vérification aussi au relâchement, pour éviter de préparer un faux nouvel appui.

50 ms est un point de départ pédagogique, également utilisé dans l’exemple Arduino Debounce. Deux lectures identiques ne prouvent pas une stabilité continue entre elles ; un appui très bref peut aussi passer inaperçu pendant l’attente.

AVR-LibC : util/delay.h. Arduino : exemple officiel Debounce. Un filtrage qui attend une stabilité observée pendant toute une durée est plus robuste que cette comparaison simple.

* Anti-rebond / debounce : filtrage matériel ou logiciel qui évite de compter les transitions temporaires d’un contact comme plusieurs actions.
* Échantillon : lecture d’un signal prise à un instant particulier, qui ne décrit pas à elle seule ce qui s’est passé entre deux lectures.

L’horloge donne le rythme ; le timer compte

L’horloge* cadence les opérations du processeur. À **16 MHz**, elle fournit 16 millions de cycles par seconde : un cycle dure **62,5 ns**. Une instruction peut demander un ou plusieurs cycles ; une ligne de C peut produire plusieurs instructions.

F_CPU décrit l’horloge au code
-DF_CPU=16000000UL définit une macro utilisée notamment par util/delay.h. Cette valeur doit correspondre à l’horloge réelle ; elle ne change pas matériellement la fréquence de la puce. UL signifie unsigned long.

Un **timer** est un compteur matériel. En comptage ascendant avec l’horloge interne, il augmente à chaque tick*. Un **prescaler*** divise l’horloge qui lui est envoyée, sans ralentir pour autant le CPU.

Diviseur N	Fréquence du timer à 16 MHz	Durée d’un tick
1	16 000 000 Hz	0,0625 µs
8	2 000 000 Hz	0,5 µs
64	250 000 Hz	4 µs
256	62 500 Hz	16 µs
1 024	15 625 Hz	64 µs

Dans ce cas : **fréquence du timer = fréquence CPU / N**, puis **durée d’un tick = N / fréquence CPU**. Le temps mesuré est le nombre de ticks multiplié par leur durée.

Compteur	Taille	Valeurs possibles
Timer0 — TCNT0	8 bits	0 à 255
Timer1 — TCNT1	16 bits	0 à 65 535
Timer2 — TCNT2	8 bits	0 à 255

En mode normal, après sa valeur maximale, le compteur repart à 0 : c’est un débordement. Les formules ci-dessus supposent une source interne connue ; un timer peut aussi compter des événements externes selon sa configuration.

Datasheet : 14.3 Timer/Counter Clock Sources ; 15.4 Timer/Counter Clock Sources ; 16 Timer/Counter0 and Timer/Counter1 Prescalers.

* Horloge : signal périodique qui donne un rythme commun aux opérations des circuits internes du microcontrôleur.
* Tick / prescaler : un tick est une impulsion reçue par le compteur, et le prescaler est le diviseur qui espace ces impulsions.

Timer0 : comparer et repartir à zéro

En mode **CTC***, Timer0 compte de 0 jusqu'à `OCR0A`, puis repart à 0. L'égalité avec la valeur de comparaison produit un **compare match** et un indicateur d'événement.

Registre / bit	Rôle
TCCR0A / WGM01	Participe au choix du mode ; WGM02:0 = 010 sélectionne CTC.
TCCR0B / CS02:0	Choisit la source d'horloge et le prescaler ; 011 sélectionne /64.
TCNT0	Contient la valeur actuelle du compteur.
OCR0A / OCR0B	Contiennent les valeurs de comparaison A et B ; A fixe le maximum en CTC.
TIFR0 / OCF0A	Contient les flags ; OCF0A signale une comparaison A réussie.
TIMSK0	Permet d'autoriser certaines interruptions du Timer0.

```
// TCCR0A: WGM01=1, WGM00=0; disconnect timer outputs.
TCCR0A = (1 << WGM01);

// TCCR0B: stop Timer0 and clear WGM02 for CTC mode.
TCCR0B = 0;

// TCNT0: start the counter from zero.
TCNT0 = 0;

// OCR0A: set TOP to 249, giving 250 ticks per period.
OCR0A = 249;

// TCCR0B: CS02:0=011 selects the clock divided by 64.
TCCR0B = (1 << CS01) | (1 << CS00);
```

Pourquoi 249 donne 1 ms
À 16 MHz avec /64, un tick dure 4 µs. Le compteur parcourt 250 valeurs, de 0 à 249 inclus : (249 + 1) × 4 µs = 1 000 µs = 1 ms.

Ces lignes configurent le compteur dans une fonction ; elles n'activent pas une interruption et n'inversent pas la LED. Pour effacer le flag* OCF0A, on écrit un 1 à ce bit de TIFR0 ; ce comportement est particulier :

```
// TIFR0 stores event flags: write 1 to clear OCF0A only.
TIFR0 = (1 << OCF0A);
```

Datasheet : 14.7.2 Clear Timer on Compare Match (CTC) Mode ; 14.9 Register Description, notamment TCCR0A, TCCR0B, OCR0A et TIFR0.

* CTC : Clear Timer on Compare Match, mode où le compteur est remis à zéro lorsque sa valeur atteint la comparaison choisie.
* Flag : bit indicateur d'un événement matériel, dont les règles de lecture et d'effacement dépendent du registre.

PB0 (ICP1) : enregistrer l’instant d’un signal

La notation **PB0 (ICP1)** signifie que PB0 possède une fonction alternative : l’entrée de capture* du Timer1. On ne mesure pas « le temps de PB0 » ; on enregistre la valeur du compteur quand un changement choisi arrive sur cette broche.

Nom	Ce que c’est	Rôle
PB0 / ICP1	Broche et fonction alternative	Recevoir le signal dont on veut capturer un front.
TCNT1	Compteur 16 bits du Timer1	Compter les ticks.
ICR1	Registre de capture 16 bits	Conserver la valeur copiée au moment du front.

Comme un chronomètre avec une touche « mémoriser »

Étape	Ce que fait le matériel
Le Timer1 tourne	TCNT1 passe par 100, 101, 102, 103...
Le front sélectionné arrive sur PB0	Le matériel copie, par exemple, 103 dans ICR1.
Le compteur continue	TCNT1 devient 104, 105, etc. ; la capture ne l’arrête pas.
Le programme lit ICR1	Il récupère l’instant enregistré, même s’il n’a pas lu PB0 au moment du front.

Deux captures permettent de mesurer une durée
Avec une première capture à 1 000 et une seconde à 1 400, il y a 400 ticks si le compteur n’a pas subi de débordement* entre les deux. À 2 MHz pour le timer : 400 × 0,5 µs = 200 µs.

Il faut configurer Timer1, la capture d’entrée* et le front choisi : LOW → HIGH ou HIGH → LOW. La durée dépend toujours de la fréquence effective du timer. Les retours du compteur à 0 doivent être pris en compte pour les mesures plus longues.

Pour ta LED, PB0 peut rester une sortie GPIO normale : utiliser PB0 ne signifie pas activer automatiquement ICP1. La capture sert à recevoir et dater un signal, pas à commander cette LED.

Datasheet : 13.3 Alternate Port Functions ; 15.6 Input Capture Unit ; 15.3 Accessing 16-bit Registers.

* Capture d’entrée : mécanisme matériel qui mémorise la valeur d’un timer lors d’un événement électrique sélectionné.
* Débordement : passage d’un compteur au-delà de sa valeur maximale, généralement suivi d’un retour à zéro.

Être prévenu, plutôt que vérifier en boucle

La **scrutation*** consiste à vérifier régulièrement un état dans la boucle principale. Une **interruption*** est une demande matérielle qui peut faire exécuter un gestionnaire spécial, puis reprendre le programme principal là où il avait été interrompu.

Mécanisme	Ce qui se passe
Boucle + lecture de PIND	Le programme va vérifier lui-même le bouton.
Interruption du bouton ou d'un timer	Un événement demande l'exécution d'un gestionnaire, si les autorisations nécessaires sont actives.
_delay_ms(20)	Le code attend environ 20 ms ; ce délai n'est pas une interruption et ne désactive pas les interruptions à lui seul.

L'idée de l'article sur les rebonds

Étape	Objectif
Surveiller les deux fronts	Reconnaître l'appui HIGH → LOW et le relâchement LOW → HIGH.
Suspendre le traitement des nouveaux événements	Éviter de compter chaque rebond immédiatement.
Attendre, par exemple, 20 ms	Laisser passer la période d'instabilité des contacts.
Compter les changements acceptés	i = 1 après l'appui ; i = 2 après le relâchement.
Compter une action complète	Une pression suivie d'un relâchement ; remettre ensuite i à 0.
Réautoriser les événements	Permettre de reconnaître la prochaine action.

Cette méthode est une description simplifiée
Les événements survenus pendant le blocage peuvent rester mémorisés dans des flags : les masquer ne les efface pas forcément. Une implémentation doit gérer ces indicateurs avant de réactiver le traitement.

Dans AVR-GCC, on écrit habituellement le gestionnaire avec `ISR(...)`, fourni par `<avr/interrupt.h>`. Si ton énoncé n'autorise que `<avr/io.h>` et `<util/delay.h>`, n'ajoute pas automatiquement cet en-tête. Un gestionnaire peut aussi être déclaré avec les attributs du compilateur : il reste un traitement spécial à configurer.

Ton exercice demande une inversion au **début de l'appui**, pas après la paire appui–relâchement. La scrutation avec un état précédent et un anti-rebond reste une autre façon de satisfaire ce comportement.

Datasheet : 11 Interrupts ; 12 External Interrupts ; 12.2.3 EIFR — External Interrupt Flag Register. AVR-LibC : avr/interrupt.h.

* Scrutation / polling : surveillance d'un état par des lectures régulières effectuées par le programme.
* Interruption : demande de traitement d'un événement qui peut détourner temporairement l'exécution vers un gestionnaire dédié.

Programme, travail et sauvegarde

Les trois mémoires de l'ATmega328P remplissent des rôles différents. La flash et l'EEPROM sont **non volatiles*** : leurs données restent sans alimentation. La SRAM est **volatile** : elle ne conserve pas ses données lorsque l'alimentation disparaît.

Mémoire	Rôle principal	Capacité	Sans alimentation
Flash	Instructions du programme et bootloader.	32 Kio	Contenu conservé.
SRAM / RAM	Données de travail et pile* pendant l'exécution.	2 Kio	Contenu perdu.
EEPROM	Données sauvegardées explicitement, comme un réglage.	1 Kio	Contenu conservé.

1 Kio = 1 024 octets. La flash contient donc 32 768 octets au total, dont une partie peut être utilisée par le bootloader. La SRAM contient 2 048 octets, et l'EEPROM 1 024 octets.

Le programme va de ton Mac à la flash

Étape	Résultat
Tu écris main.c	Du texte C lisible par un humain.
avr-gcc compile et lie le programme	Des instructions pour le microcontrôleur, dans un fichier ELF.
avr-objcopy produit main.hex	Un fichier Intel HEX qui décrit les données et leurs adresses.
avrdude communique avec le bootloader	Le bootloader installe l'application dans la flash.
Le programme démarre	Le processeur lit et exécute les instructions enregistrées.

Le nom du fichier ne suffit pas à connaître son format
Dans ton Makefile, main.bin peut contenir un ELF : l'extension imposée par l'exercice ne transforme pas le contenu en binaire brut. Un vrai binaire brut nécessiterait une conversion explicite.

Le bootloader est lui-même un petit programme de la flash. Il reçoit les données compilées de la nouvelle application, pas directement le texte de main.c. Une fois programmée, la carte peut fonctionner avec une alimentation, sans rester reliée à ton Mac.

Datasheet : 7.2 In-System Reprogrammable Flash Program Memory ; 7.3 SRAM Data Memory ; 7.4 EEPROM Data Memory.

* Non volatile : mémoire qui garde les informations enregistrées même sans alimentation électrique.
* Pile / stack : zone de travail utilisée notamment pour les appels de fonctions, les adresses de retour et certaines variables locales.

Carte alimentée : le programme travaille

La flash conserve les instructions, la SRAM* sert aux valeurs courantes et l’EEPROM* contient seulement les données qui y ont été enregistrées explicitement.

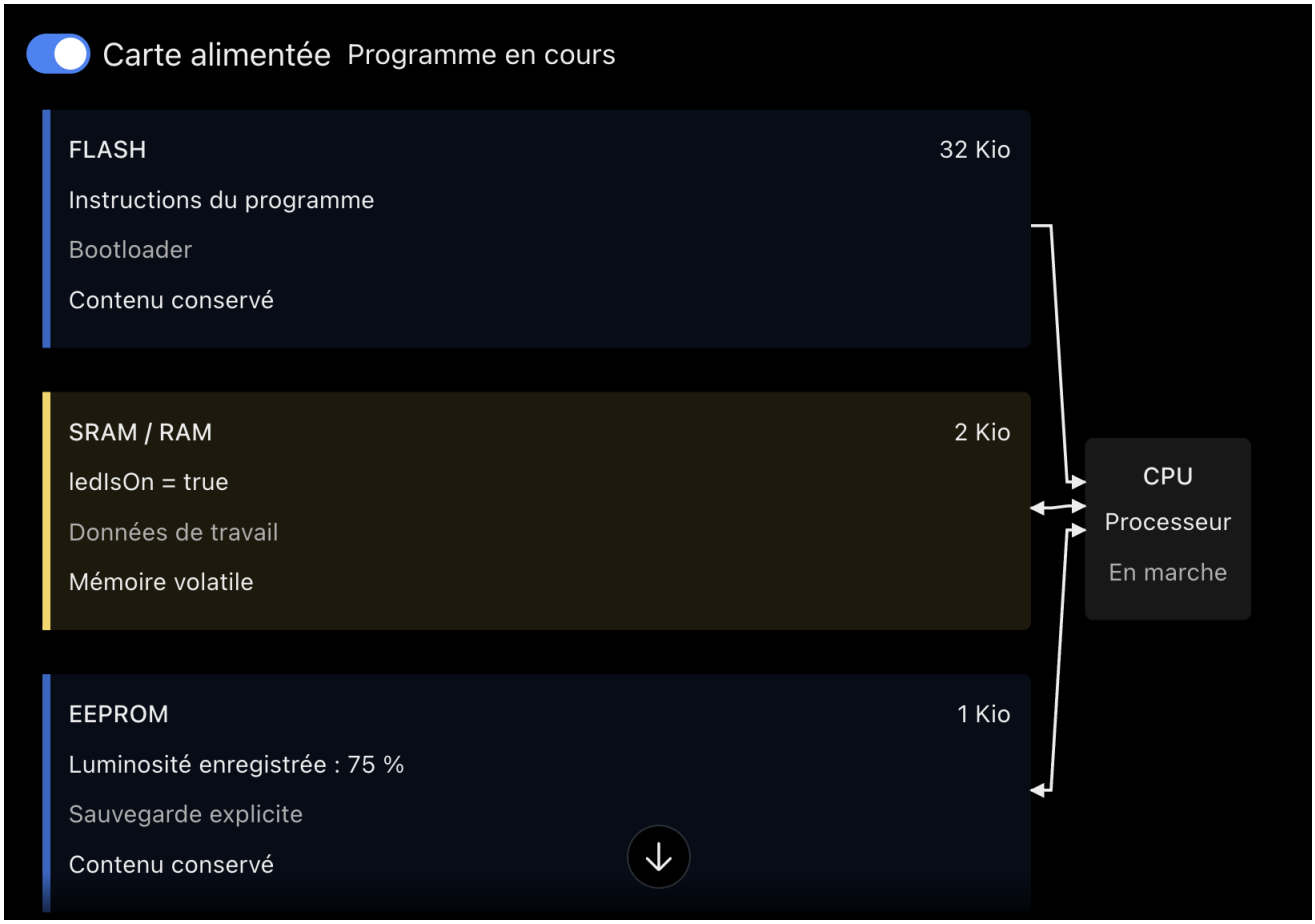


Image originale 7 — La carte fonctionne ; la variable d’exemple vaut ledIsOn = true.

Dans l’image	Ce que cela veut dire
Flash : instructions et bootloader	Le programme reste disponible pour le CPU.
SRAM : ledIsOn = true	L’état de travail peut évoluer pendant le programme.
EEPROM : luminosité 75 %	Ce réglage a été enregistré volontairement.

Une variable C n’est pas une sauvegarde automatique
Déclarer une variable normale ne la place pas en EEPROM. Selon l’optimisation, une valeur de travail peut être stockée en SRAM ou dans un registre du processeur.

Les registres GPIO, comme DDRB et PORTB, sont liés aux périphériques : ils commandent les broches. Ils ne sont pas des espaces de sauvegarde durables pour tes réglages.

* SRAM : Static Random Access Memory, mémoire de travail rapide qui doit rester alimentée pour conserver son contenu.
* EEPROM : mémoire non volatile dont le programme peut lire et écrire explicitement les données, notamment pour conserver des réglages.

Carte débranchée : qu’est-ce qui reste ?

Lors d’une coupure d’alimentation, le processeur s’arrête. Les instructions de la flash et les données sauvegardées en EEPROM restent, mais les valeurs précédentes de la SRAM ne sont plus conservées.

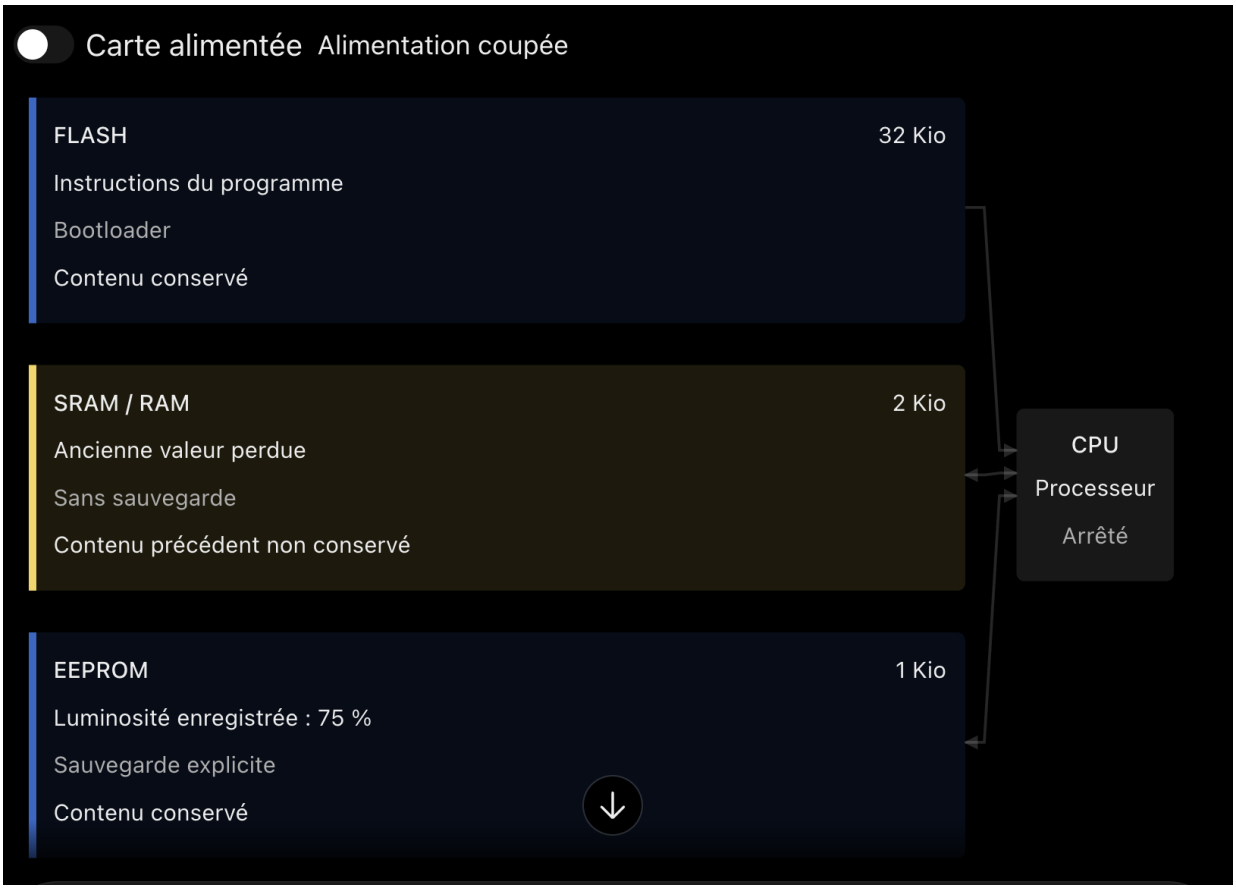


Image originale 8 — Alimentation coupée : flash et EEPROM conservées, anciennes données SRAM perdues.

Au prochain démarrage	Comportement
Le programme est toujours dans la flash.	Il peut recommencer son exécution.
Les variables sont initialisées selon le code.	Une valeur précédente n’est pas automatiquement retrouvée.
Un réglage a été sauvegardé en EEPROM.	Le code peut le relire pour le réutiliser.

Retrouver le dernier état de la LED
Il faudrait enregistrer cet état en EEPROM puis le relire au démarrage. Sinon, la LED repart selon l’initialisation* prévue par ton programme, par exemple éteinte si ledIsOn commence à false.

Les écritures non volatiles ont une endurance* limitée ; les lectures n’ont pas le même coût d’usure. Pour un réglage, on enregistre les changements utiles plutôt que d’écrire inutilement à chaque tour de boucle.

* Initialisation : attribution d’une valeur de départ prévue par le programme, au démarrage ou lors de la création d’une variable.
* Endurance : nombre de cycles d’écriture et d’effacement qu’une mémoire est conçue pour supporter.

Les sections utiles de la datasheet

Les titres ci-dessous correspondent à ta datasheet* ATmega328P **7810D–AVR–01/15**. Les pages indiquées sont celles imprimées dans le document ; une autre édition peut être organisée différemment.

Sujet	Section à chercher	Page
Flash	7.2 In-System Reprogrammable Flash Program Memory	17
SRAM	7.3 SRAM Data Memory	18
EEPROM	7.4 EEPROM Data Memory	19
Interruptions externes	12 External Interrupts	53
Choix des fronts INT0 / INT1	12.2.1 EICRA — External Interrupt Control Register A	54
Direction et pull-up	13.2.1 Configuring the Pin	59
Écriture particulière de PINx	13.2.2 Toggling the Pin	60
Lecture numérique	13.2.4 Reading the Pin Value	60
Fonctions alternatives des broches	13.3 Alternate Port Functions	63
Registre PIND	13.4.10 PIND — The Port D Input Pins Address	73
Timer0 et CTC	14.7.2 Clear Timer on Compare Match (CTC) Mode	79
Registres Timer0	14.9 Register Description	84–88
Capture ICP1 / ICR1	15.6 Input Capture Unit	95
Prescalers Timer0 / Timer1	16 Timer/Counter0 and Timer/Counter1 Prescalers	114

Documentation complémentaire

AVR-LibC — avr/io.h : noms des registres et sélection du modèle

AVR-LibC — util/delay.h : F_CPU et calcul des délais

AVR-LibC — avr/interrupt.h : gestionnaires d'interruption

Arduino — exemple Debounce, avec une durée de 50 ms

Les exemples de code de ce cahier expliquent les mécanismes ; ils ne constituent pas une solution complète de tes exercices. Les huit captures de ton document d'origine ont été conservées, sans recadrage ni modification de leur contenu.

* Datasheet : documentation technique du fabricant qui décrit les capacités, les registres et les caractéristiques électriques du composant.